

Predicting a Player's Final Ranking

from Game Statistics

Rostand Surel

Data Scientist — Paris, France

☎ 075-335-6106 | ✉ rostandsurel@yahoo.com | 🔗 linkedin.com/in/rostand-surel

🌐 github.com/Manda404 | 🌐 Portfolio | 📍 Paris, France

Abstract—In this report, I present the complete analysis and prediction pipeline that I built to estimate a player's team's normalized final ranking from Battle Royale post-match statistics. I first clarify missing-value codes, incomplete match structure, and actual feature availability. I then create business-oriented features, retain 16 variables, and keep every player from the same match in the same validation group. I compare five model families under identical conditions. CatBoost achieves the lowest mean absolute error (MAE), 0.06145, versus 0.26788 for the global-median baseline. Snapping predictions to rankings that are possible within each match gives 0.06098 out-of-fold MAE. The frozen model then achieves 0.06080 MAE on the grouped holdout. Finally, I reuse the saved model, without retraining, to generate the 5,000 submission predictions.

Index Terms—tabular regression, Battle Royale, exploratory analysis, feature engineering, grouped validation, CatBoost, data leakage

I. Problem and data

I start from the business problem. Around one hundred players take part in a match, but each row describes only one player after the match. `gameId` identifies the match and `teamId` its team; solo, duo, squad, and special modes change action context and group size. The target `winRankPercentage` is 1 for first place and 0 for last place. It depends on `maxRank`, not on `numTeams`.

The 30 train columns cover combat, mobility, resources and survival, match context, external rankings, identifiers, and the target. Train contains 50,000 rows, 30,983 `gameIds`, and 16 modes; unlabeled test contains 5,000 rows, 4,431 matches, and 10 modes. The brief assigns train to January–April 2024 and test to May 2024.

Table I
PROPERTIES OF THE SUPPLIED DATA.

	Train	Test
Player rows	50,000	5,000
Columns	30	29
Numeric columns	25	24
Categorical/text columns	5	5
Raw missing values	0	0
Exact duplicates	0	0
Distinct matches	30,983	4,431
Distinct modes	16	10

I-A Quality, sampling, and outliers

Clean does not mean reliable. Although Table I contains neither raw missing cells nor exact duplicates, it includes business conventions: `rankPts=-1` affects 38.38% of rows,

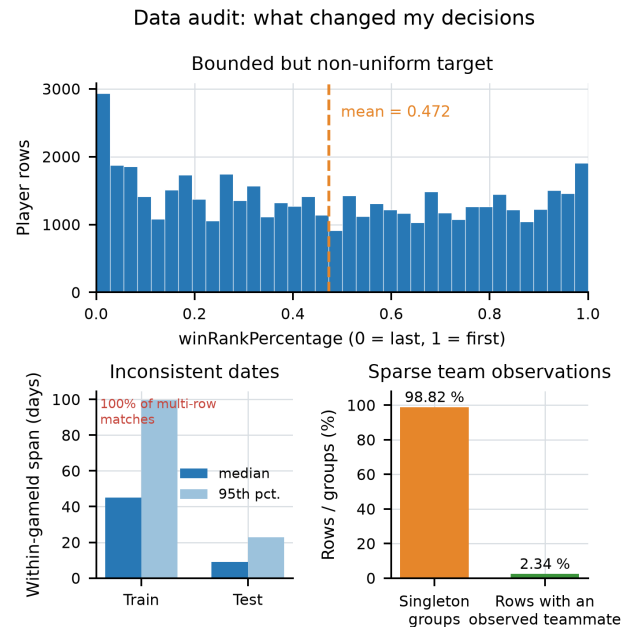


Figure 1. The target spans $[0, 1]$, but date assignment and team coverage contradict two natural assumptions. I use these findings to determine features and validation.

and conditionally zero ranking values affect 59.68%. I convert these sentinels to missing values with availability flags and exclude external rankings from the model. I group the 93 malformed `gameIds` conservatively so that the same match cannot cross partitions. I document rare gameplay inconsistencies rather than correcting them arbitrarily.

Figure 1 gives scale to these decisions. Train observes only 1.61 players per match on average, at most eight, and only 2.34% of rows have an observed teammate. In addition, the median date span within one `gameId` is 45.03 days. I therefore cannot faithfully reconstruct a team or chronology from the observed sample.

A statistical outlier is not automatically an error. The standard $1.5 \times IQR$ rule flags 24.91% of `rideDist` and 11.59% of `kills`. Both variables are concentrated at zero, so their global lower quartile is zero and the threshold becomes artificially low. When I calculate IQR only on positive values, those rates fall to 0.66% and 1.20%. I retain rare maxima because removal or winsorization would erase performance signal. I still flag the 53 rows with `kills>0` and `damages=0`, and the 102 rows with combat activity but zero distance, as



Figure 2. Mean ranking by quantile. Mobility, kills, and upgrades increase with performance, whereas killRank decreases sharply. These nonlinear shapes motivate tree-based models.

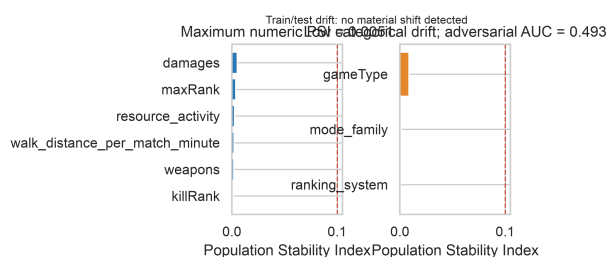


Figure 3. PSI values stay well below the 0.1 diagnostic guide, and the adversarial classifier cannot separate train from test. This supports feature comparability, not a claim about future performance.

business anomalies.

I-B Exploration and drift

The dominant signal is behavioural. The target mean is 0.4723, its median is 0.4579, and its standard deviation is 0.3074. Walking distance is the clearest behavioural signal: its Spearman correlation with the target is 0.866, and mean ranking rises from 0.093 to 0.870 from the first to the last walking-distance decile (Figure 2). I interpret this as a marker of progression and survival, not as a causal effect of walking.

Combat is also informative but less discriminative: damages and kills have correlations of 0.449 and 0.425. Resources tell a complementary story. killRank has a strong -0.715 target association, but it is computed after the match. I therefore distinguish a behavioural contract without it from an enriched post-match contract.

I also compare train and test without using a test target. Maximum numeric and categorical PSI are 0.00509 and 0.00844, while adversarial validation reaches ROC AUC 0.4933. Figure 3 therefore shows no material shift for the measured feature contract. It does not prove future performance stability, and inconsistent dates cannot support a temporal interpretation.

II. Method

II-A Validation before modeling

I define validation before comparing models. A row-random split treats players as independent, yet 56.64% of its validation

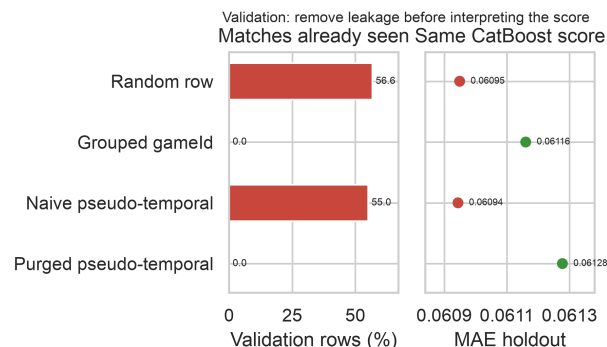


Figure 4. A similar score does not make a split valid. Grouping and purging remove seen matches; my primary protocol prioritizes valid estimation.

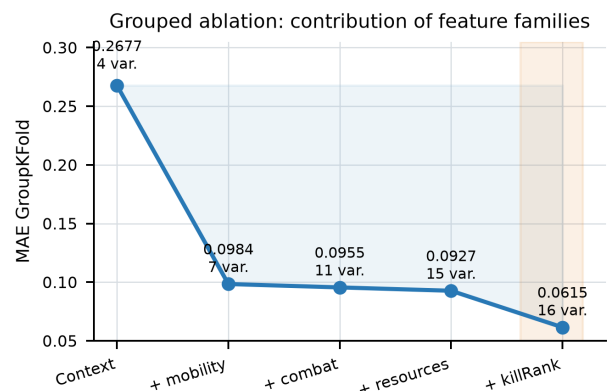


Figure 5. Progressive grouped-validation ablation. I retain each family for its reproducible gain; the final step quantifies the central role of post-match killRank.

rows come from already seen matches. I keep each match on one side of a partition. First, I freeze a 9,872-row grouped hold-out; the remaining 40,128 rows alone feed 5-fold GroupKFold, so no gameId crosses a partition.

The purged pseudo-temporal split removes leakage but drops 8,536 rows and does not repair contradictory dates. I retain it only as a stress test; grouped validation is the primary evidence. A sensitivity study over 3, 5, 7, and 10 folds confirms CatBoost in all 25 partitions. I retain five folds: about 8,026 rows per fold, while seven folds cost 52% more for only 0.000189 nominal MAE improvement.

II-B Features and model selection

I keep the contract deliberately compact. I retain eight raw measures and create four summaries: walkDist/(gameTime/60), damage per kill (zero with no kill), combat activity = kills + assists + knocks, and resource activity = weapons + upgrades + heals. Three indicators encode solo, duo, and squad, with special modes as reference. killRank is the sixteenth feature in the enriched contract.

The ablation in Figure 5 confirms that mobility provides the main behavioural gain, while combat and resources add stable signal. Without killRank, MAE is 0.09266; with it, MAE is 0.06150. Both scenarios are post-match, but the latter uses particularly strong final-match information.

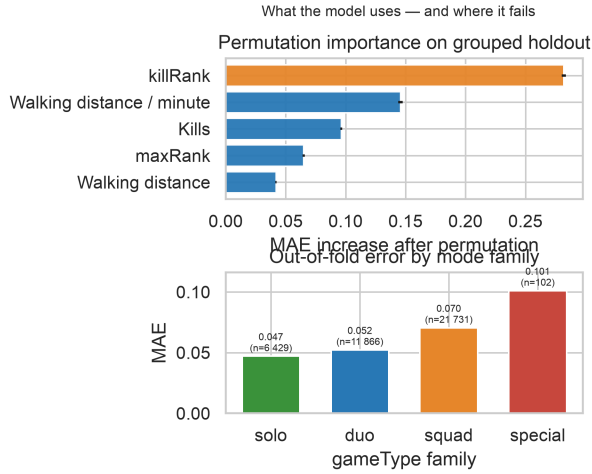


Figure 6. What the model uses and where it fails. Importance confirms the role of kill ranking and mobility; error grows in less represented, more complex modes.

I compare mean/median Dummy regressors, Ridge, Random Forest, XGBoost, LightGBM, and CatBoost on the same 16 features, rows, and folds. The primary criterion is

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|.$$

MAE measures the average ranking gap on the natural $[0, 1]$ scale, giving each player equal weight. Unlike RMSE, it is not dominated by a few extreme errors; I nevertheless inspect those errors separately. Fold stability, the train–validation gap, RMSE, and R^2 complete the decision.

III. Results, interpretation, and limitations

Table II
INITIAL COMPARISON ON FIVE GROUPED DEVELOPMENT FOLDS.

Model	MAE	σ	Train MAE	R^2
CatBoost	0.06145	0.00109	0.05527	0.92083
XGBoost	0.06345	0.00107	0.05044	0.91499
LightGBM	0.06359	0.00091	0.05949	0.91597
Random Forest	0.06668	0.00077	0.02472	0.90576
Ridge	0.08831	0.00140	0.08826	0.84177
Median Dummy	0.26788	0.00082	0.26786	−0.00294

Table II shows that CatBoost beats XGBoost in every fold by 0.00200 MAE on average and has a moderate train–validation gap. Random Forest shows the strongest overfitting. I select CatBoost *before* tuning. Eight bounded RandomizedSearchCV trials then produce a best MAE of 0.06167, 0.00022 worse than the default: I **reject** tuning and retain the simpler configuration.

The frozen model confirms performance. CatBoost reaches grouped-holdout MAE **0.06080**, RMSE 0.08660, $R^2 = 0.92083$, and a 0.00503 train–holdout gap. Permutation importance and native CatBoost TreeSHAP agree: killRank dominates, followed by walking distance per minute, kills, and maxRank. The SHAP values computed on 2,000 holdout rows reconstruct each raw prediction and describe predictive dependency, not a causal effect.

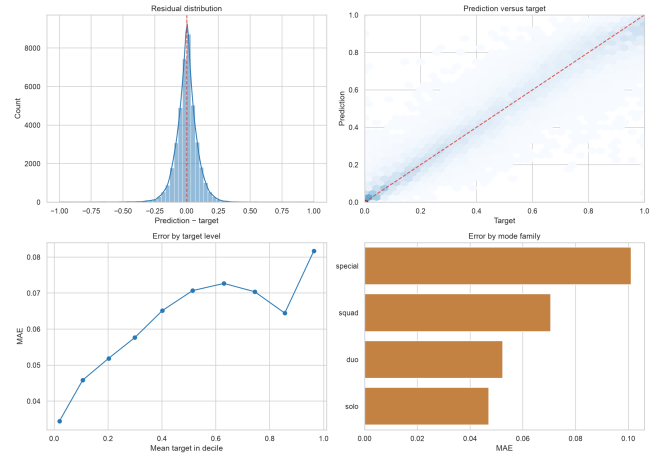


Figure 7. Grouped-holdout residual analysis. Most errors remain near zero, but performance degrades at the highest targets and in under-represented modes.

Extreme errors complement mean MAE. Out-of-fold predictions are generally centered around the target, but residuals reveal regression to the mean. On target band $[0.9, 1]$, MAE rises to 0.0811 and bias reaches -0.0755 : the best rankings are underestimated on average. Two solo cases reach absolute error 1.0 despite maxRank=18, zero kills, zero distance, and seven weapons. Missing match context, atypical rows, or label noise may explain this contrast; I retain those cases because deleting them after observing error would bias evaluation.

This result remains strictly post-match. All available statistics are post-match, inconsistent dates prevent credible future validation, rosters and matches are incomplete, and test has no target, so performance drift is unknown. A production version would require decision-time snapshots, complete teams, a coherent match date, and a genuinely labeled future batch.

Conclusion. I connect problem understanding, quality checks, outliers, exploration, validation, modeling, and interpretation. CatBoost achieves 0.06080 MAE with an explicit, reproducible grouped protocol, while the remaining extreme errors underline incomplete collective and temporal context.